

Lab Worksheet 01 - Processes

Objectives

1. Assimilate the concepts associated with the creation process, including memory management and inheritance.
2. Learn to create processes from a program and to manage basic synchronization between processes in the same family.

Submission Format

Please download the coding [canvas](#) provided for this lab assignment.

When submitting your work, please stick to the conventions described below.

You will prepare a working directory that will contain the following:

- bin directory: empty directory for all the executable files created by editing the links between your object files
- include directory: directory containing all of your header files (*.h headers)
- lib: directory containing all of your pre-compiled libraries (*.a)
- obj directory: **empty directory** for all the object files (*.o) created by compiling your source files
- src: directory containing all of your source code files (*.c)
- Makefile: file containing all of your compiler directives
- README.txt file: plain text file that reports on your work submission

Of all the elements in the working directory, the README file is the most important. It shall contain:

- Your first name, your last name, and your NYU ID
- The detailed list of files you submitted in the directories include, lib, and src (for each file, list the questions it provides a solution for)
- The explanation of the compilation rules provided in your Makefile
- Comments about the work you submitted, if any (what does not run, what is encoded but produces errors at compile / execution, what feature is incomplete, ...)
- Textual answers to some questions in the set, when applicable

Before submitting your working directory, please rename it like so:

[lastname-firstname-nyuid].os.[assignment#] (eg. marin-olivier-ogm2.os.01)

archive it in TAR format and compress it in GZIP format. The submission should be a single file:

[lastname-firstname-nyuid].os.[assignment#].tgz (eg. marin-olivier-ogm2.os.01.tgz)

Exercise 1: First process creations

Write two programs that create `N_CHILDREN` child processes.

1. The first program uses the iterative paradigm (loop)
2. The program uses recursion.

Note: This **is not** a question about process chains where a parent creates a child, the child creates a grandchild, ... **All created processes must have the same parent.**

Exercise 2: First parent/child synchronizations

Repeat exercise 1, but this time use `'wait'` primitives so that the initial process waits for:

1. the termination of exactly one child (any child)
2. the termination of the last child it created (that one only)
3. the termination of all its children

Exercise 3: Parallel compilation

Write a C program whose arguments are a list of `.c` source file paths, compiles each of them separately and simultaneously, and then edits the links to produce an executable. This program must create a child process for each file in the list. Each child runs the `gcc -c` program on the file it gets assigned. The parent awaits the termination of all its children; if all children processes terminate correctly, the parent performs the link edition by running `gcc` on all the `.o` files produced by its children.

You can test your program on the following folder: [concurrent-compilation.tgz](#)

Here's an example of how your program would get called:

```
$ ./parallel_compiler main.c titi.c tata.c toto.c
```

Exercise 4: Spy shell

Write a program `'spy'` which acts as a wrapper to the `shell` program.

Once launched, `'spy'` reads every command entered by the user (download the skeleton code ["spy-incomplete.c"](#) for code that reads user input from the terminal) and executes this command with an `exec` call.

Use `'execlp'` for commands entered with exactly one argument (eg. `'$ls -l'`), `'execvp'` for all the others.